

DIGITAL FUNDAMENTALS & COMPUTER ARCHITECTURE

MODULE 4

Logic design conventions, Building a datapath, A simple implementation scheme, An overview of pipelining - Pipelined datapath and control – Structural hazards - Data hazards - Control hazards

I/O organization - Accessing I/O devices, interrupts - handling multiple devices, Direct memory access

LOGIC DESIGN CONVENTIONS

Logic design conventions are **standard ways to represent digital circuits and signals** in processor design. They help engineers **communicate, draw, and understand complex designs** clearly.

a) Signal Representation

Binary Levels:

0 = Low voltage (logic 0)

1 = High voltage (logic 1)

Active High / Active Low:

Active High: Signal is effective when 1

Active Low: Signal is effective when 0 (often shown with a bar, e.g., RESET $\bar$)

b) Logic Gates

Logic gates are used in digital circuits because they perform **basic logical operations** that are essential for all computer processing. They take **binary inputs (0 and 1)** and produce a **binary output** based on logical rules.

- Symbols:

AND: D-shaped with two inputs, output high only if both inputs are high.

OR: Curved input shape, output high if at least one input is high.

NOT: Triangle with a bubble, inverts input.



AND

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1



OR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1



XOR

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0



NAND

A	B	Output
0	0	1
0	1	1
1	0	1
1	1	0



NOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	0



XNOR

A	B	Output
0	0	1
0	1	0
1	0	0
1	1	1

c) Timing and Clock Conventions

Processors use a **clock signal** to synchronize operations.

Rising edge / Falling edge: Operations occur at transitions of the clock ($0 \rightarrow 1$ or $1 \rightarrow 0$).

d) Bus Conventions

When drawing or understanding block diagrams of a computer system (CPU, memory, I/O), bus conventions help us interpret the signals correctly. These conventions indicate how many bits a line represents and how data flows.

- A bus carries:
 - Data** (Data Bus)
 - Addresses** (Address Bus)
 - Control signals** (Control Bus)

Single line = single bit

- **Multiple lines = bus (group of bits)**, e.g., 8-bit data bus carries 1 byte.
- **Tri-state logic:** Allows multiple devices to share a bus without conflict.

e) Register and Memory Conventions

These conventions help represent how a computer's processor and memory are organized and how instructions are formatted.

- **Registers** are small, fast storage units inside the CPU.
- They are **labeled with simple names** such as: **R0, R1, R2 ...**
- These names represent different general-purpose registers.
- Each register typically holds one word of data (8, 16, 32, or 64 bits depending on the system).
- **Memory** is organized into **locations**, and each location has a **unique address**.
- Memory locations are labeled like: **1000, 1001, 1002, ...** (decimal) or **0x00A0, 0x00A1 ...** (hexadecimal)

A machine instruction usually consists of:

- **Opcode**

The operation code

Tells the CPU *what action* to perform

Example: ADD, SUB, LOAD, STORE

- **Operand(s)**

The data items or register names involved

Could be registers or memory addresses

Example: R1, R2, 1000

- **Addressing Mode**

Addressing Mode refers to the technique used by the CPU to **identify the location of operands** (data) required for executing an instruction.

Example:

- Immediate addressing
- Direct addressing
- Indirect addressing
- Register addressing

Instruction format: Opcode + Operand(s) + Addressing mode

BUILDING A DATAPATH

Datapath:

- A **datapath** is the part of the CPU responsible for **executing instructions** and **manipulating data**.
- It consists of **functional units** like the ALU, registers, and memory, along with buses and multiplexers that **allow data to flow** between them.
- The **control unit** generates signals to coordinate operations in the datapath.

Components of a Basic Datapath

1. Instruction Memory:

- Stores instructions of the program.
- Given a memory address from the Program Counter (PC), it outputs the instruction.
- No control signals are needed for reading.

2. Program Counter (PC):

- Holds the address of the current instruction.
- Updated every clock cycle (usually $PC + 4$) to fetch the next instruction.
- No control signals needed for normal operation.

3. Adder:

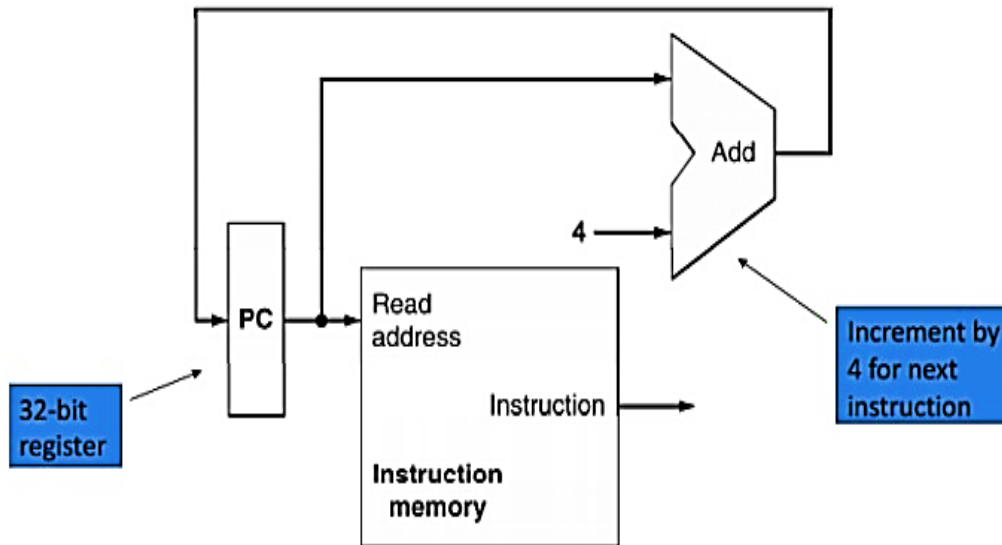
- Adds a fixed value (like 4) to the PC to get the next instruction address.
- it helps calculate the target address.

4. Register File:

- A collection of registers storing temporary data.
- Two registers can be read simultaneously; one register can be written.
- Writing requires the RegWrite control signal.

5. ALU (Arithmetic Logic Unit):

- Part of the CPU that **does all the calculations**.
- It takes two values from the **registers**.
- A **control signal** (called ALU control) tells the ALU whether to add, subtract, AND, OR, etc.



*This figure illustrates the **instruction fetch stage** of a simple single-cycle datapath*

1. Program Counter (PC)

The **PC** is a **32-bit register**.

It stores the **address of the current instruction**.

2. Instruction Memory

The PC sends its value to the **instruction memory**.

The memory uses this value as a **read address**.

Then the memory outputs the **instruction** stored at that address.

3. **PC + 4 (Increment by 4)**

Every instruction is **32 bits (4 bytes)**.

To go from one instruction to the next, PC needs to **increase by 4**.

How is this done?

The PC value goes into an **adder**.

A constant value **4** is added.

The result is **PC + 4**.

This value represents the **address of the next instruction**.

4. **Updating the PC**

The output of the adder (**PC + 4**) is sent back and written into the **PC register**.

This updates the PC so that, in the next cycle, the processor reads the **next instruction**.

Types of Elements in the Datapath

- **State Elements (Memory Elements):**

These elements **store information** and maintain state across clock cycles.

Examples: **Program Counter (PC), Instruction Memory (IM), Registers.**

They hold values until updated by a clock signal.

- **Combinational Elements:**

These elements **perform operations on inputs to produce outputs immediately**, without storing any state.

Examples: **Adder, ALU (Arithmetic Logic Unit), Multiplexers (MUX).**

Their output depends **only on the current inputs**, not on previous inputs.

Simple Implementation Scheme (Single-Cycle Datapath)

In a **single-cycle datapath**, every instruction executes in **one clock cycle**. Here's the **step-by-step flow**:

Instruction Fetch (IF)

- PC provides the address to **instruction memory**.
- Instruction is fetched and stored in **instruction register (IR)**.
- PC incremented ($PC + 4$) using **adder**.

Instruction Decode (ID)

- The instruction fetched from memory is broken into parts, such as
 - Opcode
 - source registers (rs, rt)
 - destination register (rd)
 - immediate value

ADDI R4, R4, 10
- The CPU understands what type of instruction it is (add, load, store, branch, etc.).
- The control unit generates control signals to tell the CPU how to execute the instruction.

Execution / Effective Address (EX)

- ALU performs arithmetic/logic operation or calculates memory address.
- ALU input can be a register value or immediate (selected by MUX).

Memory Access (MEM)

- For **load/store instructions**, the calculated address is sent to **data memory**.
- Data is read (load) or written (store) according to **MemRead/MemWrite** signals.

Write Back (WB)

- Results from ALU or memory are written back into the **destination register**.
- Controlled by **RegWrite** signal.

Pipelining

- Pipelining is a method used in CPUs to make instruction execution faster by doing **multiple tasks at the same time**.
- Instead of completing one instruction fully before starting the next, the CPU divides instruction execution into **smaller stages** (like Fetch, Decode, Execute, etc.).
- Each stage works on a different instruction at the same time.
- This makes the processor **much more efficient** because no part of the CPU sits idle.

Example

Think of a washing machine, dryer, and ironing table:

- Person 1 loads clothes into washing machine
- Person 2 uses dryer
- Person 3 irons clothes

After the first batch moves to the dryer, the washing machine becomes free for the next batch.

Similarly in pipelining:

- One instruction is being fetched
- Another one is being decoded
- Another is being executed
- All happening in the same clock cycle.

Why Pipelining is Useful?

1. More instructions completed in less time

- Even though each instruction still takes the same time to complete, **more instructions finish per unit time.**

2. Efficient use of hardware

- Every unit of CPU (like ALU, instruction fetch unit) keeps working continuously.

3. Higher performance without increasing clock speed

- CPU becomes faster without changing instruction design.

Hazards in Pipelining

- Pipelining allows multiple instructions to be processed **simultaneously** in different stages of the CPU.
- While this improves performance, it can also cause **problems called hazards**, which **delay or disrupt instruction execution**.
- **Hazards** occur when an instruction **cannot proceed at its scheduled time** due to resource conflicts.
- If not managed, hazards can cause **wrong results**

Types of Hazards

- Structural Hazards
- Data Hazards
- Control Hazards (Branch Hazards)

Structural Hazards

Occur when **two or more instructions require the same hardware resource at the same time.**

Cause:

- Limited hardware units such as ALU, memory, or register ports.

Example:

- If a CPU has a **single memory** used for both instruction fetch (IF) and data access (MEM), a load instruction may conflict with an instruction fetch in the same cycle.

Solution:

- **Add resources:** Use separate instruction and data memories (Harvard architecture).
- **Pipeline stall:** Delay one instruction until the resource becomes free.

Data Hazards

Occur when an instruction depends on the **result of a previous instruction that has not yet completed**.

Cause:

- In pipelining, later instructions may **read operands too early**, before the ALU or memory writes the result.

Types of Data Hazards:

- **RAW (Read After Write):** Most common. Occurs when an instruction reads a register that a previous instruction is writing to.
- **WAR (Write After Read):** Occurs when an instruction writes to a register before a previous instruction reads it. Rare in simple pipelines.
- **WAW (Write After Write):** Occurs when two instructions write to the same register in wrong order. Common in advanced pipelines.

Solutions:

- **Forwarding / Bypassing:** Pass the result of ALU directly to the dependent instruction without waiting for write-back.
- **Stalling / Bubble:** Delay dependent instruction until the required data is available.

Control Hazards (Branch Hazards)

Occur when the pipeline makes a **wrong decision on branch or jump instructions**, causing the next instruction to be incorrect.

Cause:

- CPU cannot determine the **next instruction** until the branch instruction is executed

Solution

Branch Prediction

- CPU guesses if the branch will be taken or not.
- Correct guess → pipeline continues smoothly.
- Wrong guess → discard wrong instructions and fetch the correct ones.

I/O Organization

I/O Organization explains **how the CPU communicates with external devices** such as:

- Keyboard
- Mouse
- Pen drive
- Printer

A proper I/O organization is needed to **transfer data efficiently** without wasting CPU time.

Why I/O Organization Is Important?

- Different devices work at different speeds
- CPU must know **when** a device is ready
- CPU should not waste time waiting
- Data must transfer correctly and safely
- So, a good I/O system improves both **performance** and **reliability**.

Accessing I/O Devices (Modes of Data Transfer)

I/O devices can be accessed in three primary ways:

- Programmed I/O (Polling)
- Interrupt-Driven I/O
- Direct Memory Access (DMA)

1. Programmed I/O (Polling)

Programmed I/O (PIO) is a method of transferring data between the **CPU and an I/O device** in which the CPU directly controls and manages all data transfers.

The CPU actively waits for the device to become ready and then reads or writes data.

How it Works

- **CPU Initiates the Operation:**
 - The CPU decides it wants to read data from or write data to an I/O device.
 - It accesses the **I/O device's status register** to check if the device is ready.
- **Polling the Status Register:**
 - Every I/O device has a **status register** that indicates its current state. For example:
 - 0 → Device is **not ready**
 - 1 → Device is **ready**
 - The CPU continuously **polls** (checks) this status register in a loop until the device indicates it is ready.

- **Data Transfer:**

Once the status register shows the device is ready, the CPU performs the **data transfer**:

- **Read operation:** CPU reads data from the device using the **data bus**.
- **Write operation:** CPU writes data to the device using the **data bus**.

- **CPU Active During Entire Transfer:**

Unlike DMA (Direct Memory Access), the CPU is **actively involved the entire time**:

- It keeps checking the status register.
- It controls every single byte of data transfer.

Disadvantages

- CPU time wasted in busy waiting.
- Inefficient for high-speed devices.

- **Interrupt-Driven I/O**

Interrupt-Driven I/O is a technique where the **I/O device informs the CPU** when it is ready to send or receive data, instead of the CPU constantly checking the device. The CPU **responds only when interrupted**, making the process more efficient.

How It Works

- The CPU is normally busy executing its main program.
- Each I/O device has a **status flag** to indicate readiness.
- When an I/O device is ready (e.g., data available to read or ready to accept data), it sends an **interrupt signal** to the CPU.

- The CPU **pauses its current work**, saves its execution context (like program counter, registers), and jumps to a **special routine called Interrupt Service Routine (ISR)**.
- The CPU performs the required data transfer between itself and the I/O device.
- Once the transfer is complete, the CPU **restores its previous state** and resumes the main program exactly where it left off.

Disadvantages

- Requires **extra hardware** (Interrupt Controller) and **software** (ISR).
- **Complexity**: CPU must save and restore state; priority handling for multiple interrupts can be tricky.
- Frequent interrupts can still slow down CPU if not managed properly.

Direct Memory Access (DMA)

DMA is a feature in computers that allows **certain hardware devices** (like hard disks, graphics cards, sound cards, network cards) to **transfer data directly to or from main memory (RAM) without involving the CPU** for each byte of data.

Why is DMA needed?

Without DMA:

- The **CPU must copy every piece of data** from the I/O device to memory.
- This slows down the system.
- CPU becomes busy and cannot do other tasks.

With DMA:

- Data transfer happens directly between memory and device.
- CPU is free to do other work.
- Faster, efficient input/output operations.

Operation Flow:

- CPU sets up DMA controller: source, destination, and transfer size.
- DMA controller transfers data directly between memory and I/O device.
- CPU is free for other tasks.
- DMA sends an **interrupt** to CPU when transfer is complete.

How DMA Works

1. CPU sets up DMA transfer

- CPU gives the DMA controller:
- Starting memory address
- Number of bytes to transfer
- Direction (read/write)
- Then the CPU says: “DMA, you handle this!”

2. DMA controller takes over the system bus

- DMA temporarily controls:
 - Address bus
 - Data bus
 - Control bus
- This is called **bus mastering**.

3. DMA transfers data directly

- Data goes:
- **Device → RAM** (example: disk read)
- **RAM → Device** (example: disk write)
- CPU is NOT involved in each step.

4. DMA signals completion

- After the transfer, DMA sends an interrupt:
- **DMA Interrupt**
to tell CPU: “Your data transfer is done.”

Interrupts

- An **interrupt** is a signal sent by an I/O device or software to the CPU to **attract its attention for urgent or asynchronous tasks**.
- It temporarily **pauses the CPU's current execution**, allowing it to handle the high-priority event, after which the CPU resumes its previous task.

Types of Interrupts

- **Hardware Interrupts**
- **Software Interrupts**

Hardware Interrupts

- Generated by **hardware devices** such as keyboard, mouse, printer, or network card.
- Example:
 - Keyboard sends an interrupt when a key is pressed.
 - Printer sends an interrupt when ready to accept data.
- Triggered **asynchronously**, i.e., at unpredictable times.

Software Interrupts

- Generated by **programs or software**.
- Often used to request **system services** from the operating system.

Examples:

- **Arithmetic Exception:** Division by zero triggers a software interrupt.
- **Invalid Memory Access:** Trying to access memory outside allocated range generates a software interrupt.

Bus Arbitration

When multiple devices (CPU, DMA controller, I/O devices) want to use the **system bus**, a decision must be made about **who gets control**. This decision process is called **bus arbitration**.

Below are **two commonly used bus arbitration schemes**:

1. Daisy Chain (Serial Arbitration)

How It Works

- Devices are connected in a chain.
- Bus grant signal flows from **highest priority to lowest priority device**.
- The first device that needs the bus and receives the grant gets control.

Features

- Simple and cheap to implement
- Priority is fixed based on physical order

Disadvantage

- Devices at the end of the chain may starve (low priority always)

2. Centralized Arbitration (Parallel Arbitration)

How It Works

- A **central bus arbiter** controls access.
- All devices send bus request signals to the arbiter.
- Arbiter checks priority and sends a bus grant to one device.

Features

- Fast decision making
- Can implement both fixed and rotating (round-robin) priorities
- Fairer than daisy chain

Disadvantage

- Requires additional hardware (arbiter circuit)

Interrupt Handling Steps

When an interrupt occurs, the CPU follows these steps:

- **Complete Current Instruction**
 - CPU finishes executing the current instruction to maintain program correctness.
- **Save CPU State**
 - CPU stores:
 - Program Counter (PC)
 - Registers
 - Status flags
 - Ensures the program can **resume execution correctly** after ISR.
- **Execute ISR (Interrupt Service Routine)**
 - CPU jumps to the **address of the ISR**.
 - ISR contains code to **handle the interrupt**, e.g., read data from keyboard.
- **Restore CPU State**
 - CPU restores saved context (PC, registers, flags) to resume execution.
- **Resume Main Program**
 - CPU continues execution as if no interruption occurred.

Handling Multiple Devices in Interrupt-Driven I/O

In a computer system, many I/O devices operate simultaneously—keyboard, mouse, disk, printer, network card, etc. Any of these devices may need the CPU's attention at any moment.

Therefore, the system must have a mechanism to **identify, prioritize, and service** multiple interrupts efficiently.

Interrupt handling becomes complex when *two or more devices generate interrupts at the same time* or when interrupts occur quickly one after another.

To solve this, computer systems use:

- **Priority Interrupts**
- **Interrupt Vector Table (IVT)**
- **Polling by Interrupt Controller**

Each one plays a specific role in handling multiple devices.

Priority Interrupt

- Priority interrupt is a hardware or software mechanism where each I/O device is assigned a **priority level**.
If two interrupts occur, the CPU will service the **device with higher priority first**.

Why Needed?

- Some devices require **faster response** than others.
Examples:
- A **disk controller** may need urgent service to avoid data corruption → **high priority**
- A **printer** prints slowly → **low priority**

Types of Priority Systems

- **Fixed Priority**
 - Priorities are pre-assigned.
 - Example: Keyboard > Mouse > Printer
- **Dynamic Priority**
 - Priority changes depending on workload.
 - Example: A lower device may become high priority if it has been waiting too long.

How It Works

- When multiple interrupts arrive, the **interrupt controller compares the priority levels**.
- Device with highest priority sends the interrupt signal to CPU first.
- CPU services high priority ISR and later handles lower priority interrupts.

Example

- If **keyboard** and **printer** interrupt at the same time:
- Keyboard has higher priority → CPU executes keyboard ISR.
- Printer ISR is executed afterward.

Interrupt Vector Table (IVT)

An **Interrupt Vector Table** is a special memory table that contains the **starting addresses of the Interrupt Service Routines (ISRs)** for all devices.

Why Needed?

- When an interrupt occurs, the CPU must **quickly find the correct ISR** for the device.
Instead of searching or checking each device, the CPU:
- Reads the vector number (interrupt ID)
- Uses it to index into the IVT
- Jumps directly to the correct ISR address

Structure

- The IVT contains:
- 1 entry per interrupt type
- Each entry stores: | Interrupt Number | Address of ISR |

Polling by Interrupt Controller

Polling by Interrupt Controller means that the **interrupt controller (hardware chip)** checks all devices and determines **which device sent the interrupt**.

Why Needed?

- Without polling:
- CPU would have to check all devices manually.
- This wastes time and increases overhead.

How It Works

- When interrupt occurs:
- Interrupt controller receives signals from all devices.
- It checks the **status registers** of each device.
- It identifies:
 - Which device raised the interrupt
 - Whether multiple interrupts are pending
- It sends the **interrupt ID** or **priority level** to CPU.
- CPU uses IVT to jump to correct ISR.

**THE
END**